

Le Signal Form sous stéroïdes

```
declarativeForm(  
  {  
    age: 18,  
    name: '',  
  },  
  insertField(  
    'age',  
    insertRequiredField,  
    insertValidate(({ model }) => ({  
      isValid: computed(() => model().age >= 18)  
      error: 'You must be at least 18 years old'  
    })),  
  ),  
)
```



**@craft-ng - Type-Safe Reactive States
Management for Angular**
Promotes declarative and composable
code with logic management utilities
Par Romain Geffrault



1. declarativeForm

Un formulaire basé sur le signal form d'Angular, qui gère sa logique de façon déclarative.

Pourquoi ? Parce que le signal form actuel a quelques limitations, mais est une excellente base pour construire un formulaire déclaratif qui embarque sa logique.

Ce que je pense mettre en place et qui n'est pas inclus dans le code de base du signal form actuel (je suppose) :

- Erreurs et warnings de validation type-safe : toutes les erreurs possibles seront typées et accessibles de façon déclarative.
- Gestion de la logique incluse dans le formulaire (ex : un array où on souhaite ajouter un élément)
- Le formulaire est en « readonly » et il faut utiliser les méthodes exposées pour le mettre à jour, ce qui permet de mieux contrôler les mises à jour et d'éviter les mutations directes
- S'intègre parfaitement avec les autres primitives (ex : query pour charger les données initiales, mutation pour soumettre le formulaire, validations asynchrones, possibilité d'utiliser des insertions existantes comme `insertLocalStoragePersister`)
- Gestion des formulaires en parallèle (ex : plusieurs formulaires sur la même page, cas des mutations granulaires, une liste d'entités avec un formulaire par entité pour les mettre à jour en parallèle)

Je commence à réfléchir à ce que cela peut donner en termes d'API, voici une ébauche.



Cas "simple"

```
const myForm = declarativeForm(
  {
    age: 18,
    name: '',
  },
  insertField(
    'age',
    insertRequiredField,
    insertValidate(({ model }) => ({
      isValid: computed(() => model().age >= 18),
      error: 'You must be at least 18 years old',
    })),
  ),
  insertField(
    'name',
    insertDebounceValidation(
      300,
      insertRequiredField,
      insertMinLengthField(3),
    ),
  ),
  ({ set, initialValue }) => ({
    reset: () => set(initialValue),
  }),
);
// 'You must be at least 18 years old' | 'required' | undefined
myForm.selectAge().exceptions();
// 'required' | 'minLength3' | undefined
myForm.selectName().exceptions();
myForm.reset(); // resets the form to its initial value
```



Créer un utilitaire match ?

Peut-être que ce genre de logique de validation est assez courante pour mériter un utilitaire dédié ? Un `match` qui prend une liste de règles et retourne la première qui correspond ?

Voir aussi pour un `match.multiples` pour les cas où on veut retourner plusieurs erreurs/warnings à la fois

```
insertValidate(({ model, insertions: { debouncedModel } }) =>
  match({
    tooYoung: {
      isValid: computed(() => model().age > 18),
      error: 'You are too young',
    },
    tooOld: {
      isValid: computed(() => model().age < 120),
      error: 'You are too old',
    },
  }),
),
```



Mettre des warnings type-safe ?

```
insertField(
  'email',
  insertRequiredField,
  insertWarnings(({ model, initialValue }) => ({
    isValid: computed(
      () => initialValue.email && model().email !== initialValue.email
    ),
    warn: 'Changer cet email déconnectera tous vos appareils.',
  })),
);

// 'Changer cet email déconnectera tous vos appareils.' | undefined
myForm.selectEmail().warnings();
```



async validation

```
const checkUserStatus = query({
  method: (user: User) => user,
  loader: async ({ params: user }) => {
    // return a User or CommonHttpException
    // (ex: like disconnected device error)
    return checkUserStatus(user);
  },
});

const myForm = declarativeForm(
  {
    id: '',
    age: 18,
    name: '',
  } satisfies User,
  // ...
  insertAsyncValidate({
    checkUserStatus: asyncValidate(checkUserStatus, {
      onSuccess: ({ result }) => {
        return result.status === 'banned'
      }
    },
    ? { isValid: false, error: 'Your account is banned' } : unde
  },
  {
    },
  },
  /// ...
);

// { checkUserStatus: 'Your account is banned' | CommonHttpException
myForm.exceptions();
```



submit

```
const saveUser = mutation({
  // can only be called by the form that validate the user data
  // which guarantees that the data is valid when this method is
  method: (user: Validated<User>) => user,
  loader: async ({ params: user }) => {
    // return a User or CommonHttpException (ex: like disconnected)
    return patchUser(user);
  },
});

const myForm = declarativeForm(
  {
    id: '',
    age: 18,
    name: '',
  } satisfies User,
  insertSubmit(saveUser, {
    onSuccess: ({ setAsInitialValue, model }) => {
      setAsInitialValue(model);
    },
    // exceptions returned by the mutations
    // will be automatically added to the form exceptions
  }),
  /// ...
);

myForm.exceptions(); // CommonHttpError | ...
```



parallel form/submit 1/2

```
const saveUser = mutation({
  method: (user: Validated<User>) => user,
  identifier: ({ params }) => params.id,
  loader: async ({ params: user }) => {
    // return a User or CommonHttpException
    // (ex: like disconnected device error)
    return patchUser(user);
  },
});

const checkUserStatus = query({
  method: (user: User) => user,
  identifier: ({ params }) => params.id,
  loader: async ({ params: user }) => {
    return checkUserStatus(user);
  },
});
```



parallel form/submit 2/2

```
const myParallelForms = parallelDeclarativeForm(
  {
    id: '1',
    age: 31,
    name: 'Romain',
  } satisfies User,
  insertForm({
    // Distinguishes parallel forms
    identifier: ({ model }) => model().id,
  },
  insertAsyncValidate({
    checkUserStatus: asyncValidate(checkUserStatus, {
      onSuccess: ({ result }) => checkIsBanned(result),
    }),
  }),
  insertField(
    'name', //...
  ),
  ({ update }) => ({
    // Methods to add/remove forms in parallel
    addForm: (user: User) => { ... },
    removeForm: (userId: string) => { ... },
  }),
);

myParallelForms.addForm({ id: '2', age: 18, name: 'John' });
// { checkUserStatus: 'Your account is banned' | CommonHttpExc
// | { name: 'required' | 'minLength3' } | ...
myParallelForms.selectForm('1').exceptions();
```



form with array

```
const myForm = declarativeForm(
  {
    id: '',
    age: 18,
    name: '',
    ideas: [] as Idea[],
  } satisfies User,
  insertFormTree('ideas', {
    insertField(
      'title',
      insertRequiredField,
      insertMinLengthField(3),
    ),
    ({ update, model }) => ({
      addIdea: (idea: Idea) => { ... },
      removeIdea: (ideaId: string) => { ... },
      total: computed(() => model().ideas.length),
    })
  }),
  /// ...
);

myForm.selectIdeas().total; // number
myForm.selectIdeas().addIdea({ id: '1', title: 'New idea' });
myForm.selectIdeas().removeIdea('1');
```



interdependent field logic

Validation interdépendante entre plusieurs champs du formulaire.

Par exemple, pour des dates d'ouverture et de fermeture, la date d'ouverture doit être antérieure à la date de fermeture, et inversement.

Si la date d'ouverture est modifiée et que la date de fermeture n'a pas été modifiée manuellement,

on peut automatiquement mettre à jour la date de fermeture pour qu'elle soit 10 jours après la date d'ouverture.



interdependent field logic - example

```
const myForm = declarativeForm(
  {
    openedDate: null as Date | null,
    closedDate: null as Date | null,
  },
  insertField('openedDate', // ...),
  insertField(
    'closedDate',
    insertRequiredField,
    insertValidate(({ model }) => ({
      isValid: computed(() => {
        const openedDate = model().openedDate;
        const closedDate = model().closedDate;
        return isClosedDateAfterOpenedDate(openedDate, closedDate);
      }),
      error: 'Closed date must be after opened date',
    })),
    ({ model: { openedDate }, closedDateField, set }) => ({
      _setClosedDate10DaysAfterOpenedDate: afterRecomputation(
        openedDate,
        () => {
          if (closedDateField.dirty()) {
            return;
          }
          set(addDays(openedDate(), 10));
        },
      ),
    }),
  ),
);
// 'Closed date must be after opened date'
// | 'required' | undefined
myForm.selectClosedDate().exceptions();
```



Conclusion

Je trouve cette première ébauche d'API assez prometteuse pour construire un formulaire déclaratif par composition.

Gérer le pilotage de la logique aussi facilement que les validations est un grand atout.

Je suis peut-être passé à côté de quelque chose. N'hésitez pas à me faire part de vos retours et idées d'amélioration !

N'hésitez pas à me dire si cela vous intéresse, ça m'aidera à me motiver car il y a pas mal de travail pour construire une telle API.



**Merci pour la lecture !
Dis-moi ce que t'en
penses en commentaire.**



Romain Geffrault
Développeur Angular
Suis moi pour du contenu dédié à
Angular